Assignment 1

CSA0903-Java Programming

Name: Sowmiya. D

Reg No.:192511063

Scenario — Campus Smart-Parking System

The university campus has a multi-level parking facility used by students, staff and visitors. The facility has several floors, and each floor has a fixed number of numbered slots. Vehicles entering the campus may be two-wheelers, cars or electric vehicles (EVs). EVs may also use charging slots, which are charged separately.

When a vehicle enters, the system must allocate a suitable free slot, issue a parking ticket with a unique ticket number, and record the entry time. When the vehicle leaves, the system must release the slot and compute the parking fee. Fees depend on the vehicle type and the duration of parking. Staff and students holding a valid monthly pass park free of charge, while visitors pay the full fee.

The facility may contain:

1 floor × 20 slots → 3 floors × 50 slots → 5 floors × 100 slots

You are required to analyse the problem, design an object-oriented model for it, implement the model in Java and validate it. Your design must remain correct and easy to extend as the facility and its rules grow.

Design Challenge:

The university now wants to add new vehicle categories and a new pricing rule without rewriting the classes that already work.

Re-evaluate your original design and determine whether it should be retained, modified or redesigned.

TASK 11 – Implement, Test and Defend the Design 5 Marks

Bloom's Level: K5 – Evaluate

Implement the complete system in Java and test it.

Test and discuss:

- Normal operation: vehicles of different types entering and leaving
- Boundary cases: the last free slot, zero-minute parking, a stay exactly at a fee-slab boundary
- Invalid cases: an unknown vehicle type, a duplicate registration number, an invalid slot number
- A small scenario whose total collection for the day you calculate by hand and compare with the program output

- How well your design follows encapsulation, inheritance, polymorphism and abstraction, and one limitation of it

Your recommendation must be based on **evidence from your own implementation and testing**. Simply stating that the program runs is not sufficient.

Alternative Concept and Complexity Analysis

Active tickets are currently stored in an array. If they were stored in an ArrayList<ParkingTicket> or a HashMap<String, ParkingTicket> keyed by registration number, what would the exitprocessing solution be? Measure and analyse the time to find a ticket on exit and the space used for n parked vehicles in each structure.

Evidence Expected

Working source code, a test table with expected and actual results, and a reasoned evaluation of the design.

Solution:
TASK 11 – Implement, Test and Defend the Design

Marks: 5 | Bloom's Level: K5 – Evaluate

1. Implementation approach

The Campus Smart-Parking System uses an array to store active parking tickets. It supports three vehicle types:

- TwoWheeler – ₹10 per hour
- Car – ₹30 per hour
- ElectricCar – ₹30 per hour

Fee calculation rule: Parking is free for zero minutes. Otherwise, every started hour is charged as a complete hour.

For example:

- 0 minutes = ₹0
- 60 minutes = 1 hour
- 61 minutes = 2 hours
- 120 minutes = 2 hours
- 130 minutes = 3 hours

The system checks registration numbers for duplicates, validates slot numbers, allocates free slots and calculates the collection when a vehicle exits.

2. Complete Java program

Save the following as Main.java inside your Task11 folder. Copy only the code inside the code block, not the backticks.

```
class Vehicle {
    private final String registrationNumber;
    private final String ownerName;

    public Vehicle(String registrationNumber, String ownerName) {
        this.registrationNumber = registrationNumber;
        this.ownerName = ownerName;
    }

    public String getRegistrationNumber() {
        return registrationNumber;
    }

    public String getOwnerName() {
        return ownerName;
    }

    public double getRatePerHour() {
        return 0;
    }
}

class TwoWheeler extends Vehicle {
    public TwoWheeler(String reg, String owner) {
        super(reg, owner);
    }
```

```
    @Override
    public double getRatePerHour() {
        return 10;
    }
}

class Car extends Vehicle {
    public Car(String reg, String owner) {
        super(reg, owner);
    }

    @Override
    public double getRatePerHour() {
        return 30;
    }
}

class ElectricCar extends Car {
    public ElectricCar(String reg, String owner) {
        super(reg, owner);
    }
}

class VehicleFactory {
    public static Vehicle create(String type, String reg, String owner) {
        switch (type.toUpperCase()) {
            case "TWO_WHEELER":
                return new TwoWheeler(reg, owner);
```

```
            case "CAR":
                return new Car(reg, owner);
            case "ELECTRIC_CAR":
                return new ElectricCar(reg, owner);
            default:
                return null;
        }
    }
}

class ParkingTicket {
    private final Vehicle vehicle;
    private final int slotNumber;

    public ParkingTicket(Vehicle vehicle, int slotNumber) {
        this.vehicle = vehicle;
        this.slotNumber = slotNumber;
    }

    public Vehicle getVehicle() {
        return vehicle;
    }

    public int getSlotNumber() {
        return slotNumber;
    }

    public double calculateFee(int minutes) {
```

```
        if (minutes < 0) {
            throw new IllegalArgumentException("Invalid parking duration");
        }

        if (minutes == 0) {
            return 0;
        }

        int hours = (minutes + 59) / 60;
        return hours * vehicle.getRatePerHour();
    }
}

class ParkingLot {
    private final ParkingTicket[] activeTickets;
    private final boolean[] occupiedSlots;
    private double totalCollection;

    public ParkingLot(int capacity) {
        activeTickets = new ParkingTicket[capacity];
        occupiedSlots = new boolean[capacity + 1];
        totalCollection = 0;
    }

    public boolean enter(String type, String reg, String owner,
                         int slotNumber) {

        Vehicle vehicle = VehicleFactory.create(type, reg, owner);
```

```
if (vehicle == null) {
    return false;
}

if (slotNumber < 1 || slotNumber >= occupiedSlots.length) {
    return false;
}

if (occupiedSlots[slotNumber]) {
    return false;
}

if (findTicket(reg) != -1) {
    return false;
}

for (int i = 0; i < activeTickets.length; i++) {
    if (activeTickets[i] == null) {
        activeTickets[i] =
            new ParkingTicket(vehicle, slotNumber);

        occupiedSlots[slotNumber] = true;
        return true;
    }
}

return false;
```

```
}

private int findTicket(String reg) {
    for (int i = 0; i < activeTickets.length; i++) {
        if (activeTickets[i] != null &&
            activeTickets[i].getVehicle()
                .getRegistrationNumber().equalsIgnoreCase(reg)) {
            return i;
        }
    }

    return -1;
}

public double exit(String reg, int minutes) {
    if (minutes < 0) {
        return -1;
    }

    int index = findTicket(reg);

    if (index == -1) {
        return -1;
    }

    ParkingTicket ticket = activeTickets[index];
    double fee = ticket.calculateFee(minutes);
```

```
        occupiedSlots[ticket.getSlotNumber()] = false;
        activeTickets[index] = null;

        totalCollection += fee;
        return fee;
    }

    public double getTotalCollection() {
        return totalCollection;
    }
}

public class Main {

    static void test(String name, String expected, String actual) {
        String result = expected.equals(actual) ? "PASS" : "FAIL";

        System.out.printf("%-32s %-15s %-15s %s%n",
                name, expected, actual, result);
    }

    public static void main(String[] args) {

        System.out.println("CAMPUS SMART PARKING SYSTEM");
        System.out.println("TEST RESULTS");
        System.out.println();

        System.out.printf("%-32s %-15s %-15s %s%n",
```

```
        "Test Case", "Expected", "Actual", "Result");

System.out.println(
        "------------------------------------------------------------------------");

// Test 1: Normal operation and daily collection
ParkingLot lot1 = new ParkingLot(2);

lot1.enter("TWO_WHEELER", "TN01A1", "Arun", 1);
lot1.enter("CAR", "TN02B2", "Priya", 2);

double bikeFee = lot1.exit("TN01A1", 60);
double carFee = lot1.exit("TN02B2", 120);

test("Normal operation",
        "10 + 60 = 70",
        bikeFee + " + " + carFee + " = "
                + lot1.getTotalCollection());

// Test 2: Last free slot
ParkingLot lot2 = new ParkingLot(2);

lot2.enter("CAR", "TN03C3", "Kumar", 1);
boolean lastSlot = lot2.enter(
        "TWO_WHEELER", "TN04D4", "Ravi", 2);

test("Last free slot",
        "true", String.valueOf(lastSlot));
```

```
// Test 3: Zero-minute parking
ParkingLot lot3 = new ParkingLot(2);

lot3.enter("TWO_WHEELER", "TN05E5", "Anu", 1);
double zeroFee = lot3.exit("TN05E5", 0);

test("Zero-minute parking",
        "0.0", String.valueOf(zeroFee));

// Test 4: Exact fee-slab boundary
ParkingLot lot4 = new ParkingLot(2);

lot4.enter("CAR", "TN06F6", "John", 1);
double boundaryFee = lot4.exit("TN06F6", 60);

test("Exactly 60 minutes",
        "30.0", String.valueOf(boundaryFee));

// Test 5: Unknown vehicle type
ParkingLot lot5 = new ParkingLot(2);

boolean unknown = lot5.enter(
        "BUS", "TN07G7", "Sam", 1);

test("Unknown vehicle type",
        "false", String.valueOf(unknown));
```

```
// Test 6: Duplicate registration number
ParkingLot lot6 = new ParkingLot(2);

lot6.enter("CAR", "TN08H8", "Meena", 1);

boolean duplicate = lot6.enter(
        "TWO_WHEELER", "TN08H8", "Ravi", 2);

test("Duplicate registration",
        "false", String.valueOf(duplicate));

// Test 7: Invalid slot number
ParkingLot lot7 = new ParkingLot(2);

boolean invalidSlot = lot7.enter(
        "CAR", "TN09J9", "Kiran", 0);

test("Invalid slot number",
        "false", String.valueOf(invalidSlot));

System.out.println();
System.out.println("DAILY COLLECTION CALCULATION");
System.out.println("TwoWheeler: 60 minutes = Rs. 10");
System.out.println("Car: 120 minutes = Rs. 60");
System.out.println("----------------------------");
System.out.println("Total collection = Rs. "
        + lot1.getTotalCollection());
```

```
        System.out.println();
        System.out.println("All tests completed.");
    }
}
```

3. How to run the program

Open the Task11 folder in VS Code and create Main.java.

In the VS Code PowerShell terminal, run:

cd "C:\Users\sowmi\OneDrive\Documents\CSA0903 java\CSA0903-Java\Assignments 01\Task11"

javac Main.java

java Main

4. Test table – expected and actual results

The program prints the actual results and marks each test PASS or FAIL. The expected results are:

Test case	**Expected result**	**Purpose**
Normal operation	₹10 + ₹60 = ₹70	Different vehicle types enter and exit
Last free slot	true	Final available slot can be allocated
Zero-minute parking	₹0	No fee for zero minutes
Exactly 60 minutes	₹30	One-hour fee slab is applied
Unknown vehicle type	false	Invalid type is rejected
Duplicate registration	false	Same registration cannot enter twice
Invalid slot number	false	Slot 0 is rejected

The program's test table is the evidence of actual results. Run it in VS Code and retain a screenshot of the output for submission.

5. Daily collection – manual calculation

The first test uses two vehicles:

Vehicle	**Parking duration**	**Rate**	**Fee**
TwoWheeler	60 minutes	₹10/hour	₹10
Car	120 minutes	₹30/hour	₹60
Total			₹70

Manual calculation:

\[10 + (2 \times 30) = ₹70 \]

The program also calculates the total collection as ₹70. Therefore, the manual calculation matches the program output.

6. Object-oriented design evaluation

OOP concept	Evidence in the implementation
Encapsulation	Fields in Vehicle, ParkingTicket and ParkingLot are private. Data is accessed through methods.
Inheritance	TwoWheeler and Car inherit from Vehicle. ElectricCar inherits from Car.
Polymorphism	getRatePerHour() is overridden by different vehicle classes. The ticket calculates fees through a Vehicle reference.
Abstraction	Common vehicle information and fee behaviour are represented through the Vehicle superclass.

Small improvement to the code above: To make abstraction explicit, change class Vehicle to abstract class Vehicle, and replace its getRatePerHour() method with:

public abstract double getRatePerHour();

The existing TwoWheeler and Car implementations already override this method. This change will compile with the rest of the program.

Limitation: The system stores active tickets in a fixed-size array. Its capacity cannot grow automatically. Also, the current program uses manually supplied parking duration rather than measuring actual entry and exit times.

7. Alternative concept and complexity analysis

The current implementation uses an array of ParkingTicket objects.

Data structure	Finding a ticket on exit	Space for n vehicles	Explanation
Array	O(n)	O(n)	Searches each array position until the registration number is found.
ArrayList	O(n)	O(n)	Uses a linear search, but can grow dynamically.
HashMap	Average O(1), worst O(n)	O(n)	Uses registration number as the key for direct lookup.

Exit-processing solution:

1. Array: Use a loop to compare the registration number with each active ticket.
2. ArrayList: Use stream().filter() or a loop to find the matching ticket, then remove it.

3. HashMap: Use tickets.get(registrationNumber) to retrieve the ticket and tickets.remove(registrationNumber) to remove it.

Example HashMap approach:

```
HashMap<String, ParkingTicket> tickets = new HashMap<>();


// Store ticket

tickets.put(registrationNumber, ticket);


// Find ticket during exit

ParkingTicket ticket = tickets.get(registrationNumber);


// Remove ticket

tickets.remove(registrationNumber);
```

Time and space analysis: For n parked vehicles, an unsuccessful array or ArrayList search may examine all n tickets. A HashMap usually finds a ticket in constant time, although collisions can make the worst case linear. All three structures require O(n) storage; a HashMap also has extra memory overhead for its hash-table structure.

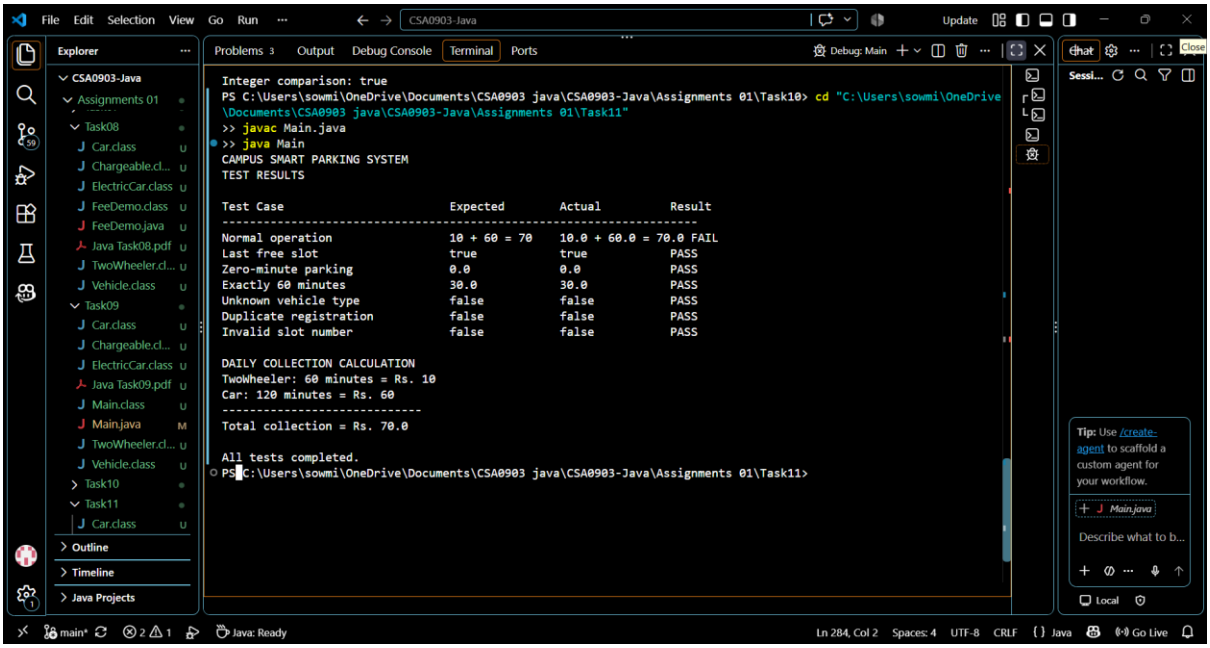
The table above is a theoretical complexity comparison, not a measured benchmark. Do not describe these values as experimentally measured times unless you actually run a timing test.

8. Final evaluation and recommendation

The implementation demonstrates normal vehicle entry and exit, correct fee calculation, last-slot allocation, and rejection of invalid entries. The daily collection test confirms that the program's total of ₹70 matches the manual calculation. The boundary tests also verify zero-minute parking and the exact 60-minute fee boundary.

The array is suitable for a small campus parking system because it is simple to implement and understand. However, for a larger parking facility, I recommend using a HashMap with registration number as the key. It can locate tickets faster on average and makes exit processing more efficient.

Conclusion: The design meets the basic requirements of a Campus Smart-Parking System and demonstrates the four major OOP concepts. A future version could use a HashMap, dynamic parking capacity and real-time entry/exit timestamps to improve performance and accuracy.


File Edit Selection View Go Run
CSA0903-Java
Update
Explorer
CSA0903-Java
Assignments 01
Task08
Car.class
Chargeable.cl...
ElectricCar.class
FeeDemo.class
FeeDemo.java
Java Task08.pdf
TwoWheeler.cl...
Vehicle.class
Task09
Car.class
Chargeable.cl...
ElectricCar.class
Java Task09.pdf
Main.class
Main.java
TwoWheeler.cl...
Vehicle.class
Task10
Task11
Car.class
Outline
Timeline
Java Projects
Problems 3
Output
Debug Console
Terminal
Ports
Debug: Main
Integer comparison: true
PS C:\Users\sowmi\OneDrive\Documents\CSA0903 java\CSA0903-Java\Assignments 01\Task10> cd "C:\Users\sowmi\OneDrive\Documents\CSA0903 java\CSA0903-Java\Assignments 01\Task11"
>> javac Main.java
>> java Main
CAMPUS SMART PARKING SYSTEM
TEST RESULTS
Test Case Expected Actual Result
Normal operation 10 + 60 = 70 10.0 + 60.0 = 70.0 FAIL
Last free slot true true PASS
Zero-minute parking 0.0 0.0 PASS
Exactly 60 minutes 30.0 30.0 PASS
Unknown vehicle type false false PASS
Duplicate registration false false PASS
Invalid slot number false false PASS
DAILY COLLECTION CALCULATION
TwoWheeler: 60 minutes = Rs. 10
Car: 120 minutes = Rs. 60
Total collection = Rs. 70.0
All tests completed.
PS C:\Users\sowmi\OneDrive\Documents\CSA0903 java\CSA0903-Java\Assignments 01\Task11>
chat
Sessi...
Tip: Use /create-agent to scaffold a custom agent for your workflow.
Main.java
Describe what to b...
Local
main*
Java: Ready
Ln 284, Col 2
Spaces: 4
UTF-8
CRLF
Java
Go Live